

ЛАБОРАТОРНА РОБОТА № 1

Принципи програмування. DRY, KISS, SOLID, YAGNI та інші

Мета: навчитися дотримуватися принципів програмування та обґрунтовувати їх.

Хід роботи:

Завдання 0: Підготовка до виконання завдання.

1. Створити окремий публічний репозиторій на GitHub. В цьому репозиторії будуть міститися всі виконані Вами завдання курсу “Конструювання програмного забезпечення”. Кожне завдання буде міститися в окремій директорії *lab-1*, *lab-2* ... *lab-n*.
2. Впевнитися, що Ви уважно прочитали пункт 1 і Ваш репозиторій точно публічний ;)
3. Склонувати створений репозиторій.
4. Створити директорію *lab-1*.
5. Обрати один з трьох варіантів Завдання 1 ☺

Результат виконання:

1. Створюємо окремий публічний репозиторій на GitHub, в якому будуть міститися всі виконані завдання курсу “Конструювання програмного забезпечення”.

					ДУ «Житомирська політехніка».25.121.08.000 – Лр1			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Дзінзілевич Д.О.			Звіт з лабораторної роботи		Літ.	Арк.
Перевір.		Фант М.О.						1
Керівник							ФІКТ Гр. ІПЗ-23-2[1]	
Н. контр.								
Зав. каф.								

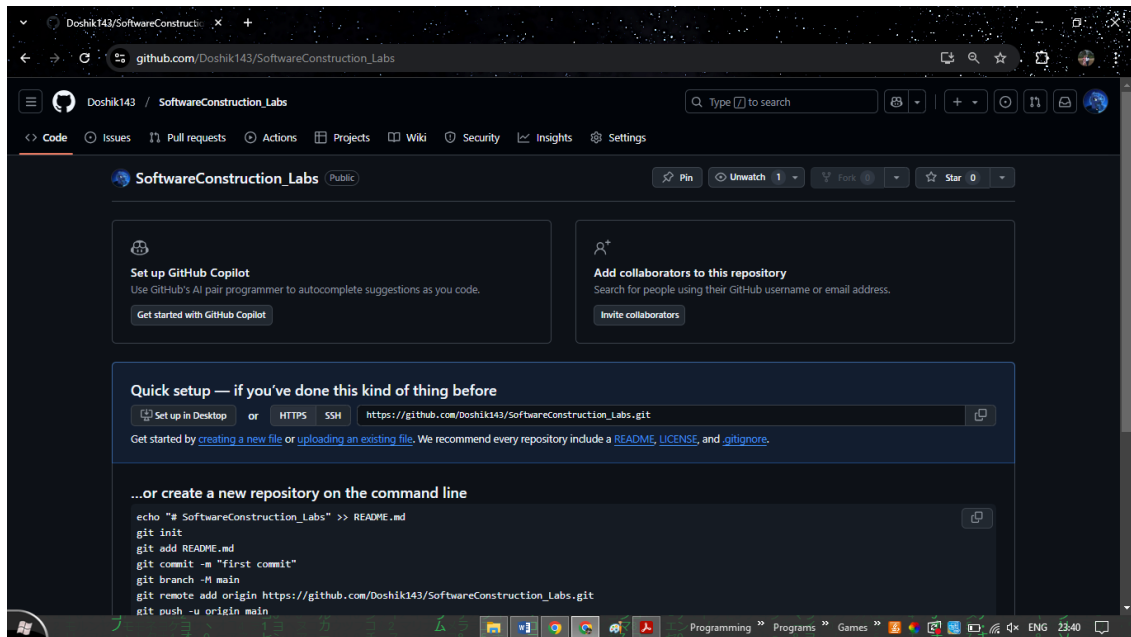


Рис.1.1 Створений репозиторій

2. Перевіряємо чи точно репозиторій є публічним. І як бачимо, так, він є публічним.

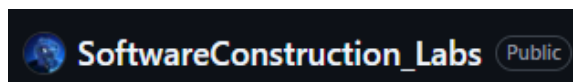


Рис.1.2 Позначення, що репозиторій публічний

3. Клонуємо створений репозиторій на комп'ютер.

```
C:\Users\ipz23>cd source\repos
C:\Users\ipz23\source\repos>git clone https://github.com/Doshik143/SoftwareConstruction_Labs.git
Cloning into 'SoftwareConstruction_Labs'...
warning: You appear to have cloned an empty repository.
```

Рис.1.3 Клонування репозиторію на комп'ютер

4. Створюємо директорію *lab-1*.

```
C:\Users\ipz23\source\repos>cd SoftwareConstruction_Labs
C:\Users\ipz23\source\repos\SoftwareConstruction_Labs>mkdir lab-1
```

Рис.1.4 Створення необхідної директорії

5. Для Завдання 1 обираємо перший варіант.

Завдання 1 (**Варіант 1**): Виконати завдання з дотриманням відомих Вам принципів програмування.

- Запрограмуйте клас **Money** (об'єкт класу оперує однією валютою) для роботи з грошима. У класі мають бути передбачені: поле для

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Пр1	Арк.
		Фант М.О.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

зберігання цілої частини грошей (долари, євро, гривні тощо) і поле для зберігання копійок (центи, євроценти, копійки тощо). Реалізувати методи виведення суми на екран, задання значень частин.

2. Створити клас **Product** для роботи з продуктом або товаром. Реалізувати метод, який дозволяє зменшити ціну на задане число.
3. Реалізувати клас **Warehouse**, який описує товари, що зберігаються на складі: найменування, одиниця виміру, ціна одиниці, кількість, дата останнього завантаження, тощо.
4. Реалізувати клас **Reporting** для роботи зі звітністю. Реєстрація надходження товару (формування прибуткової накладної) і відвантаження (видаткова накладна). Звіт по інвентаризації (залишки на складі).
5. Для кожного з класів реалізувати необхідні методи і поля. Для класів передбачити реалізацію конструкторів та методів для встановлення та читання значень.
6. Ви також можете додавати власний функціонал для унаочнення принципів програмування. Приклади додаткового функціоналу:
 - а. категорії для продуктів;
 - б. конкретні дочірні класи валюти;
 - в. корзина для замовлень.

Результат виконання:

1. Створюємо клас **Money** для роботи з грошима.

Лістинг програми:

```
using System;

namespace ClassLibrary
{
    public class Money
    {
        public int Dollars { get; set; }
        public int Cents { get; set; }

        public Money(int dollars, int cents)
        {
            Dollars = dollars;
            Cents = cents;
        }

        public void DisplayAmount()
        {
```

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Пр1	Арк.
		Фант М.О.				3
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        Console.WriteLine($"{Dollars}.{Cents:00}");
    }
}

```

2. Створюємо клас **Product** для роботи з продуктом або товаром.

Лістинг програми:

```

using System;

namespace ClassLibrary
{
    public class Product
    {
        public string Name { get; set; }
        public Money Price { get; set; }
        public ProductCategory Category { get; set; }

        public Product(string name, Money price, ProductCategory category)
        {
            Name = name;
            Price = price;
            Category = category;
        }

        public void ReducePrice(int amount)
        {
            Price.Dollars -= amount;
        }
    }
}

```

3. Створюємо клас **Warehouse**, який описує товари, що зберігаються на складі.

Лістинг програми:

```

using System;

namespace ClassLibrary
{
    public class Warehouse
    {
        public string Name { get; set; }
        public string Unit { get; set; }
        public Money Price { get; set; }
        public int Quantity { get; set; }
        public DateTime LastDeliveryDate { get; set; }
        public ProductCategory Category { get; set; }

        public Warehouse(string name, string unit, Money price, int quantity, DateTime lastDeliveryDate, ProductCategory category)
        {
            Name = name;
            Unit = unit;
            Price = price;
            Quantity = quantity;
            LastDeliveryDate = lastDeliveryDate;
            Category = category;
        }
    }
}

```

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Пр1	Арк.
		Фант М.О.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

4. Створюємо клас **Reporting** для роботи зі звітністю.

Лістинг програми:

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace ClassLibrary
{
    public class Reporting
    {
        private List<Warehouse> _warehouseItems = new List<Warehouse>();

        public void RegisterIncomingProduct(Warehouse product)
        {
            _warehouseItems.Add(product);
            Console.WriteLine($"Product {product.Name} added to the warehouse.");
            Console.WriteLine("-----");
        }

        public void ShipProduct(string productName)
        {
            var product = _warehouseItems.FirstOrDefault(p => p.Name == productName);
            if (product != null)
            {
                _warehouseItems.Remove(product);
                Console.WriteLine($"Product {productName} shipped from the warehouse.");
                Console.WriteLine("-----");
            }
        }

        public void InventoryReport()
        {
            Console.WriteLine("Inventory Report:\n");
            foreach (var item in _warehouseItems)
            {
                Console.WriteLine($"{item.Name} - {item.Quantity} {item.Unit}\nCategory:
{item.Category}\nPrice per unit: {item.Price.Dollars}.{item.Price.Cents:00}\nLast Deliv-
ery: {item.LastDeliveryDate.ToShortDateString()}\n-----");
            }
        }
    }
}
```

5. Для кожного з класів реалізуємо необхідні методи, конструктори і поля.

а. Money.cs

- Поля:

- Dollars (ціла частина грошей)
- Cents (дробова частина грошей)

- Конструктори:

- Money(int dollars, int cents)

- Методи:

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Лр1	Арк.
		Фант М.О.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

- DisplayAmount() – виводить суму у форматі DD.CC

b. Product.cs

- Поля:

- Name (назва продукту)
- Price (ціна продукту)
- Category (категорія продукту)

- Конструктори:

- Product(string name, Money price, ProductCategory category)

- Методи:

- ReducePrice(int amount) – зменшує ціну продукту

c. Warehouse.cs

- Поля:

- Name (назва товару)
- Unit (одиниця виміру)
- Price (ціна одиниці товару)
- Quantity (кількість товарів на складі)
- LastDeliveryDate (дата останнього завозу товару)
- Category (категорія товару)

- Конструктори:

- Warehouse(string name, string unit, Money price, int quantity, DateTime lastDeliveryDate, ProductCategory category)

- Методи:

- Поля доступні через get/set властивості, що дозволяє читати їх або встановлювати значення

d. Reporting.cs

- Поля:

- _warehouseItem (список товарів на складі)

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Лр1	Арк.
		Фант М.О.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

- Конструктори:

- Reporting() – ініціалізує список _warehouseItem

- Методи:

- RegisterIncomingProduct(Warehouse product) – додає товар до списку
- ShipProduct(string productName) – видаляє товар зі списку
- InventoryReport() – виводить звіт про всі товари на складі

6. З додаткового функціоналу додамо категорії для продуктів.

Лістинг програми:

```
using System;
namespace ClassLibrary
{
    public enum ProductCategory
    {
        Food,
        Electronics,
        Clothing
    }
}
```

Завдання 2: Написати код для тестування отриманої функціональності.

1. Покажіть правильність роботи свого коду запустивши його в головному методі програми.
2. Достатньо буде просто вивести певну інформацію, щоб показати, що класи комунікують певним чином між собою.

Результат виконання:

1. Пишемо код для тестування нашої програми.

Лістинг програми:

```
using System;
using ClassLibrary;
namespace TestProgram
{
    class Program
    {
        private static void Main(string[] args)
        {
            //Creation of objects
            Money price = new Money(10, 50);
            Product product = new Product("Apple", price, ProductCategory.Food);
        }
    }
}
```

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Пр1	Арк.
		Фант М.О.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        Warehouse warehouse = new Warehouse("Apple", "kg", price, 100, DateTime.Now,
        ProductCategory.Food);
        Warehouse laptop = new Warehouse("Laptop", "pcs", new Money(1500, 0), 10,
        DateTime.Now, ProductCategory.Electronics);
        Warehouse tShirt = new Warehouse("T-Shirt", "pcs", new Money(20, 0), 50,
        DateTime.Now, ProductCategory.Clothing);
        Reporting reporting = new Reporting();

        Console.WriteLine("-----");

        //Registration of goods receipt
        reporting.RegisterIncomingProduct(warehouse);

        //Output of the inventory report
        reporting.InventoryReport();

        //Reduction in the price of the product
        product.ReducePrice(2);
        Console.WriteLine("-----");
        Console.WriteLine($"New price for {product.Name}: ");
        product.Price.DisplayAmount();

        Console.WriteLine("-----");

        //Shipment of goods
        reporting.ShipProduct("Apple");
        reporting.RegisterIncomingProduct(laptop);
        reporting.RegisterIncomingProduct(tShirt);

        //Output of the updated inventory report
        reporting.InventoryReport();

        Console.WriteLine("\n\n\nPress any key to exit...");
        Console.ReadKey();
    }
}

```

2. Перевіряємо роботу коду виведенням необхідної інформації.

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Лр1	Арк.
		Фант М.О.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

Результат виконання програми:

```
C:\Users\vipz23\source\repos\SoftwareConstruction_Labs\lab-1\TestProgram\bin\Debug\TestProgram.exe
-----
Product Apple added to the warehouse.
-----
Inventory Report:
Apple - 100 kg
Category: Food
Price per unit: 10.50
Last Delivery: 28.02.2025
-----
New price for Apple: 8.50
-----
Product Apple shipped from the warehouse.
-----
Product Laptop added to the warehouse.
-----
Product T-Shirt added to the warehouse.
-----
Inventory Report:
Laptop - 10 pcs
Category: Electronics
Price per unit: 1500.00
Last Delivery: 28.02.2025
-----
T-Shirt - 50 pcs
Category: Clothing
Price per unit: 20.00
Last Delivery: 28.02.2025
-----
Press any key to exit...
```

Рис.1.5 Результат виконання програми

Завдання 3: Опишіть особливості дотримання принципів програмування в вашому коді.

1. Додайте файл README.md в кореневу директорію цієї лабораторної роботи. В файлі README.md опишіть дотримання окремо кожного принципу програмування, який Вам відомо, і який можна продемонструвати Вашим кодом.
2. Опис можна залишати українською або (бажано) англійською мовами.
3. Ваш опис повинен містити посилання на відповідні файли і рядки коду.
4. Як залишати посилання на свої рядки коду можна глянути [тутечки](#) (для посилання на директорію) або [тут](#) (для посилання на окремі рядки).
5. Синтаксис .md файлів документації можна знайти [туть](#) або [туть](#).

6. Для отримання максимальної оцінки Ви повинні продемонструвати мінімум 7 принципів. SOLID принципи рахуються окремо. Повний список принципів, які було розглянуто на лекції:

- a. DRY;
- b. KISS;
- c. SOLID (5 окремих принципів);
- d. YAGNI;
- e. Composition Over Inheritance;
- f. Program to Interfaces not Implementations;
- g. Fail Fast.

Результат виконання:

1. Додаємо документацію в README.md зі всією необхідною інформацією.

Лістинг програми:

```
# Лабораторна робота №1

## Принципи програмування

### DRY (Don't Repeat Yourself)
- Принцип DRY дотримується у класі
[Money](https://github.com/Doshik143/SoftwareConstruction Labs/blob/SC Labs/lab-1/ClassLibrary/Money.cs), де методи для роботи з грошима не повторюються.

### KISS (Keep It Simple, Stupid)
- Клас
[Product](https://github.com/Doshik143/SoftwareConstruction Labs/blob/SC Labs/lab-1/ClassLibrary/Product.cs) реалізований просто, без зайвої складності.

### SOLID
- **Single Responsibility Principle (SRP)**: Кожен клас має одну відповідальність.
- **Open/Closed Principle (OCP)**: Класи [Money](https://github.com/Doshik143/SoftwareConstruction Labs/blob/SC Labs/lab-1/ClassLibrary/Money.cs), [Product](https://github.com/Doshik143/SoftwareConstruction Labs/blob/SC Labs/lab-1/ClassLibrary/Product.cs), [Warehouse](https://github.com/Doshik143/SoftwareConstruction Labs/blob/SC Labs/lab-1/ClassLibrary/Warehouse.cs) можна розширювати, не змінюючи їхній базовий код.
- **Liskov Substitution Principle (LSP)**: Принцип дотримується, оскільки всі класи можуть бути замінені своїми підкласами без зміни поведінки програми.
- **Interface Segregation Principle (ISP)**: Принцип дотримується, оскільки інтерфейси не перевантажені методами.
```

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Пр1	Арк.
		Фант М.О.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

- ****Dependency Inversion Principle (DIP)****: Принцип дотримується, оскільки класи залежать від абстракцій, а не від конкретних реалізацій.

YAGNI (You Aren't Gonna Need It)

- У коді не реалізовано зайвих функцій, які не потрібні для виконання завдання.

Composition Over Inheritance

- У коді використовується композиція (наприклад, клас `[Product]` (https://github.com/Doshik143/SoftwareConstruction_Labs/blob/SC_Labs/lab-1/ClassLibrary/Product.cs#L5-L22) містить об'єкт класу ``Money``).

Program to Interfaces not Implementations

- У коді використовуються [інтерфейси для забезпечення гнучкості] (https://github.com/Doshik143/SoftwareConstruction_Labs/blob/SC_Labs/lab-1/ClassLibrary/Reporting.cs).

Fail Fast

- У коді передбачено [перевірки на помилки] (https://github.com/Doshik143/SoftwareConstruction_Labs/blob/SC_Labs/lab-1/ClassLibrary/Reporting.cs#L18-L27), щоб програма не продовжувала роботу з некоректними даними.

Завдання 4: UML діаграма.

1. Підготувати діаграму створених у програмі класів та інтерфейсів за допомогою <https://app.diagrams.net/>.
2. Експортувати створену діаграму та запустити експортований файл у кореневу директорію цієї лабораторної.

Результат виконання:

1. Створюємо діаграму класів, де:

- —> - асоціація
- -----> - композиція
- —◆ - залежність

		Дзінзілевич Д.О.			ДУ «Житомирська політехніка».25.121.08.000 – Лр1	Арк.
		Фант М.О.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

Діаграма класів:

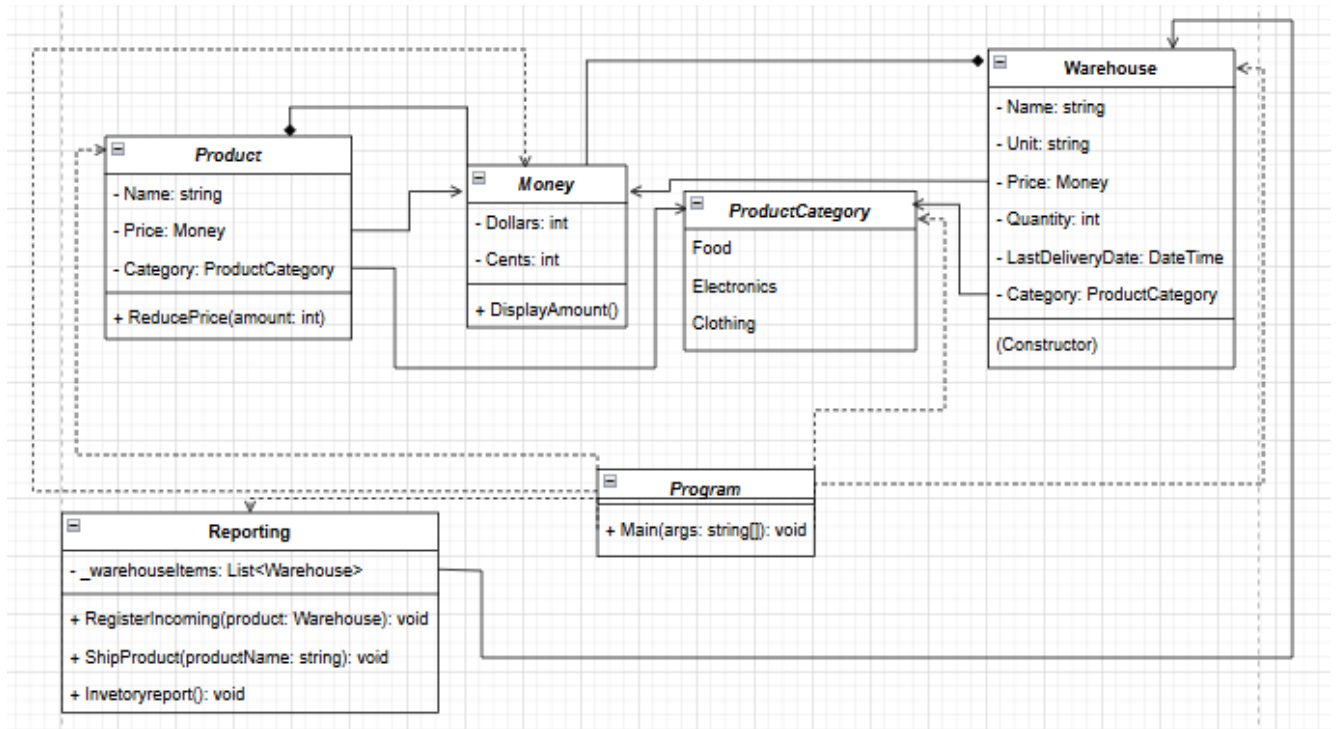


Рис.1.6 Діаграма класів

Висновки: навчилися дотримуватися принципів програмування та обґрунтовувати їх.